

Deadline. The homework is due **11:59pm, Mon 06/08, 2026**. You must submit your solutions (in pdf format generated by LaTeX) via GradeScope.

Late Policy. You have up to four grace days (calendar day) for the entire quarter. You don't lose any point by using grace days. If you decide to use grace days, please specify how many grace days you would like to use and the reason at the beginning of your submission.

Academic Integrity. You can discuss with your classmates or use online resources. In particular,

- **Collaboration Policy.** You can get help from the instructor, TAs, or your classmates, but only after you have thought about the problems on your own. **If you discussed with anyone, you need to acknowledge them in your report.**
- **Other sources.** It is OK to get inspiration (but not solutions) from books or online resources, again after you have carefully thought about the problems on your own. **If you use any reference or webpage, or discussed with anyone, you must cite it fully and completely in your report.**
- **AI Tools.** If you use ChatGPT or similar AI tools, you have to attach the full conversation to make it clear what help you obtained from them.

However, you **CANNOT**:

- share your report/code with anyone else,
- read other's report/code,
- copy anything from other source,
- get help from others without acknowledging them in your report, or use any relevant resources (e.g., online article/AI tools such as ChatGPT) without citing them.

Otherwise it will be considered cheating. We reserve the right to deduct points for using such material beyond reason. **In summary, you must write up your solution independently, and type your answers word by word on your own: close the book/notes/online resources when you are writing your final answers. Add all the citations and collaborators' names to the end.**

Write-up. Please use LaTeX to prepare your solutions. For all problems, **please explain how you get the answer** instead of directly giving the final answer, except for some special cases (will be specified in the problem). For all algorithm design problems, please describe using **natural language**. You can present pseudocode if you think that helps to illustrate your idea, but please do not only give some code without explanation. **In grading, we will reward not only correctness but also clarity and simplicity. To avoid losing points for hard-to-understand solutions, you should make sure your solutions are either self-explanatory or contains good explanations.** See some more details here <https://www.cs.ucr.edu/~yihans/teaching/214/S26/assignments/>

Bonus questions. Bonus questions are not required, and the points are not counted in the basic 100 points of the class. Most of the bonus questions are just slightly harder than the regular questions, but some can be extremely hard. In a nutshell, most of them are worthy trying, but don't be frustrated if you cannot solve *all* of them :) We will not provide too many hints/solutions for bonus problems, but feel free to discuss them with your classmates.

In this project, we will implement a parallel BFS algorithm. The input will be a graph with a source node label s . Your task is to compute the BFS distance of all vertices to s .

1 Overview

You can find the template files by this invitation link:

<https://classroom.github.com/a/rj3qo0ls>

There are two main files in the repository:

- **BFS.cpp**. This file is an example testing file for you. The actual testing code will be very similar, but differs in the choice of the graphs and sources. This file includes a checker to help you verify the correctness of your algorithm.
- **BFS.h**. You need to implement your BFS function in this file. Of course you can add auxiliary functions or variables, but please make sure to put all your code in this file.

When you test your own code, you can modify any files as you need, but **we will only keep the BFS.h in the actual test, so please make sure it can compile if keeping all other files unchanged. Also, please remember to push the changes to GitHub.**

In this project, you need to submit:

- A midterm report on May 18. In the midterm report, you need to report your current progress. The requirement of your mid report is to at least implement a purely sparse or dense mode, or anything that is runnable (probably not highly optimized).
- A final report on June 08. In the final report, you need to report how you implemented the entire algorithm, what test data you designed and some testing result.
- Your implementation of function BFS in **BFS.h**.

The goal of your algorithm is to compute the BFS distance of all vertices to s , storing them in the array `dist[]`.

Input graph format

The input graphs used for testing are stored in

`/usr/local/cs214/graph/`

The graphs we will use are `com-orkut.sym.bin`, and `soc-LiveJournal1.sym.bin` and `RoadUSA.sym.bin`. `RoadUSA` is used only for the bonus parts. The parameters are shown as follows:

Graph	n	m
com-orkut	3,072,627	234,370,166
soc-LiveJournal1	4,847,571	68,993,773
RoadUSA	23,947,348	57,708,624

We also provide a graph `sample.sym.bin` to help you debug. This graph is very small. You can uncomment the block in **BFS.h** to output the actual graph in adjacency lists format.

2 Possible Optimizations and Hints

Here are some optimizations and hints you can consider in your implementation. Of course, you can use more.

1. Coarsening. Use the similar idea as shown in Homework 0 and 1. Try to find the best threshold to determine a sequential or a parallel run. For graph algorithms, many substeps may require coarsening. For example, if a frontier is too small, you can process all vertices sequentially. Similarly, if a vertex has only a few neighbors, when processing its neighborhood, you can process all its neighbors sequentially, instead of in parallel.
2. Try to avoid using `std::vector`, especially avoid resizing it in parallel. This is because resizing involves allocating and deallocating memory, which is also slow in parallel. Use `new` or `malloc` to allocate arrays instead.
3. If you need random numbers, try to design a hash function to generate random numbers instead of `rand()`. This is because this function is not designed for running in parallel.
4. Directional optimization. Recall that we mentioned the dense representation and directional optimization in class. They will be very useful for BFS on social networks.
5. You can test different ways to switch between sparse and dense modes. That may affect the performance a lot.
6. When you debug, it may help to keep track of the running time per round. If any round is running very slowly, you could zoom into that round and debug more carefully.
7. You should also generate toy examples to test the correctness of your algorithm. Generating graphs are usually much faster than reading graphs so it would also be debugging-friendly.
8. Of course you can think of more interesting optimizations!

3 Testing

In the final test, we will use five source nodes from com-orkut and five source nodes from LiveJournal1. The scores will depend on the running time of your code. We will give you more detailed grading basis after 1-2 weeks.

4 (bonus) On graphs with larger diameters

As we can see in class, the span of BFS is proportional to the diameter of the graph. When the input graph has a larger diameter, it's hard to exploit parallelism. In this bonus task, you will try to design algorithms for graphs with large diameters. Your algorithm does not need to be the same with the previous one. You can implement a separate algorithm just for this task. In your report, please describe your algorithm and also include some performance test if necessary.

If you are using a different algorithm for the bonus part, please specify in your report how to test your code.

Your score will be up to 2pts from the tests, and 1 pt from the report. Detailed grading basis will be given after 1-2 weeks.

5 Background and References

Part of the background introduction and some useful references are from a previous programming competition on parallel BFS. You can find more details here:

<https://fastcode.org/events/fastcode-challenge/spe4ic/#parallel-breadth-first-search>

Parallel BFS has also been widely studied, even in many recent research papers. Most of the state-of-the-art parallel BFS implementations are under a frontier-based framework. Starting from a source s , the algorithm will visit all the 1-hop neighbors of s in parallel, storing them in the first frontier $\mathcal{F}_1$. In the subsequent rounds, we will always visit all vertices in the current frontier $\mathcal{F}_i$, and generate the next frontier $\mathcal{F}_{i+1}$ by finding all unvisited neighbors for all vertices in the current frontier $\mathcal{F}_i$. This process is repeated until all vertices have been visited. A simple pseudocode is shown below:

Algorithm 1: Framework of Parallel BFS

Input: A graph $G = (V, E)$ and a source $s \in V$

```
1 parallel_for_each  $v \in V$  do
2    $visited[v] \leftarrow false$ 
3  $\mathcal{F}_0 = \{s\}$ 
4  $visited[s] \leftarrow true$ 
5  $i \leftarrow 0$ 
6 while  $\mathcal{F}_i \neq \emptyset$  do
7   parallel_for_each  $v \in \mathcal{F}_i$  do
8      $visited[v] \leftarrow true$ 
9     Put all unvisited neighbors of  $v$  to  $\mathcal{F}_{i+1}$ 
10   $i \leftarrow i + 1$ 
```

In this case, all the i -hop neighbors of s will be visited in parallel in round i . The rounds are performed one after the other.

To update the `visited` array while avoiding duplicates, an atomic operation `compare_and_swap` is usually used.

In practice, you can use only two rolling frontiers instead of maintaining all of them. One of the technical challenges here is to choose an efficient way to maintain the frontier. We mentioned some potential approaches in class, and you can also find the original papers/open-source code below as references.

Useful techniques and their references

Here we provide the original paper/code sources for some useful techniques in case you are interested.

Directional optimization was introduced in [1].

In class we also discussed several ways to maintain the frontier. A commonly-used idea is introduced in Ligra [2]. The hash-table-like idea was introduced in [3].

Here I also list some state-of-the-art parallel graph processing libraries. All of them have their own implementation for BFS. You can use them as references:

- GBBS: <https://github.com/ParAlg/gbbs>
- Ligra: <https://github.com/jshun/ligra>
- Galois: <https://github.com/IntelligentSoftwareSystems/Galois>
- GAPBS: <https://github.com/sbeamer/gapbs>
- PASGAL: <https://github.com/ucrparlay/PASGAL>

Each of them are based on a series of research papers. You can find these relevant papers in the document of each library.

References

- [1] Scott Beamer, Krste Asanović, and David Patterson. Direction-optimizing breadth-first search. *Scientific Programming*, 21(3-4):137–148, 2013.
- [2] Julian Shun and Guy E Blelloch. Ligra: a lightweight graph processing framework for shared memory. In *Proceedings of the 18th ACM SIGPLAN symposium on Principles and practice of parallel programming*, pages 135–146, 2013.
- [3] Letong Wang, Xiaojun Dong, Yan Gu, and Yihan Sun. Parallel strong connectivity based on faster reachability. In *ACM Special Interest Group on Management of Data (SIGMOD)*, 2023.